

Lab3: Implementing CI/CT with Jenkins

SQT 1033 – Software Testing Tools

Rahul Shailesh Kawa (C0907637)

Instructor: Gagandeep Minhas

April 2, 2024

Implementing CI/CT with Jenkins

Introduction

The popular open-source automation server Jenkins makes applying Continuous Integration and Continuous Delivery (CI/CD) or Continuous Testing (CI/CT) methodologies easier. The objectives of this lab are to install Jenkins and set up an automated build and test pipeline for a Git-based project.

Prerequisites

- Windows/Mac/Linux access
- GitHub Account with a project to build & test (<https://github.com/raulkawa/ecommm.git>)
- JDK 17 or latest
- Git

Setup

Installation

To utilize Jenkins, it needs to be installed. We will follow these steps to install and set up Jenkins.

1. **Install JDK:** From Oracle website (<https://www.oracle.com/ca-en/java/technologies/downloads/>) for your platform.

JDK Development Kit 22 downloads

JDK 22 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

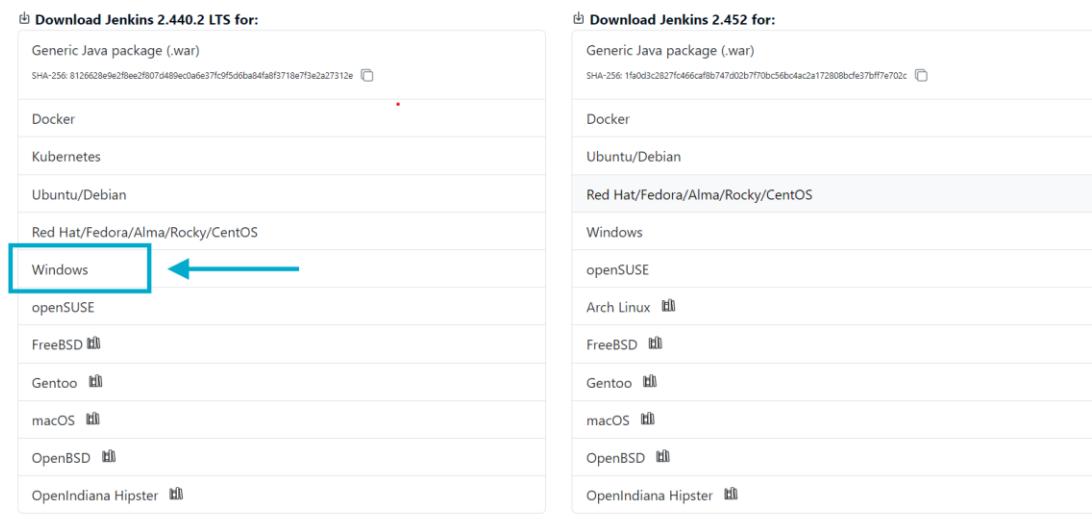
JDK 22 will receive updates under these terms, until September 2024, when it will be superseded by JDK 23.

Linux macOS **Windows**

Product/file description	File size	Download
x64 Compressed Archive	185.52 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.zip (sha256)
x64 Installer	163.91 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.exe (sha256)
x64 MSI Installer	162.07MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.msi (sha256)

JDK Installation

2. **Get Jenkins:** For your chosen platform, get the most recent Jenkins installer version from the official Jenkins website (<https://www.jenkins.io/download/>).



Jenkins Download Page

3. **Installation Method:** Following the instructions, we will complete the installation after downloading it.



Jenkins Installation

4. **Launch Jenkins:** After installation is finished, Jenkins will be started. After launching this, we can go to <http://localhost:8080> to visit the Jenkins web interface.

Initial Setup: We'll finish the initial setup and install the default plugins by following the on-screen instructions.

The following screen will appear when we browse to <http://localhost:8080> to retrieve the default password.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

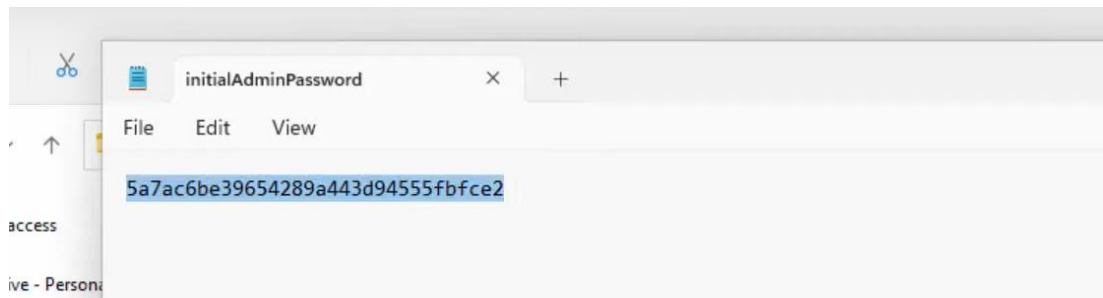
`C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`

Please copy the password from either location and paste it below.

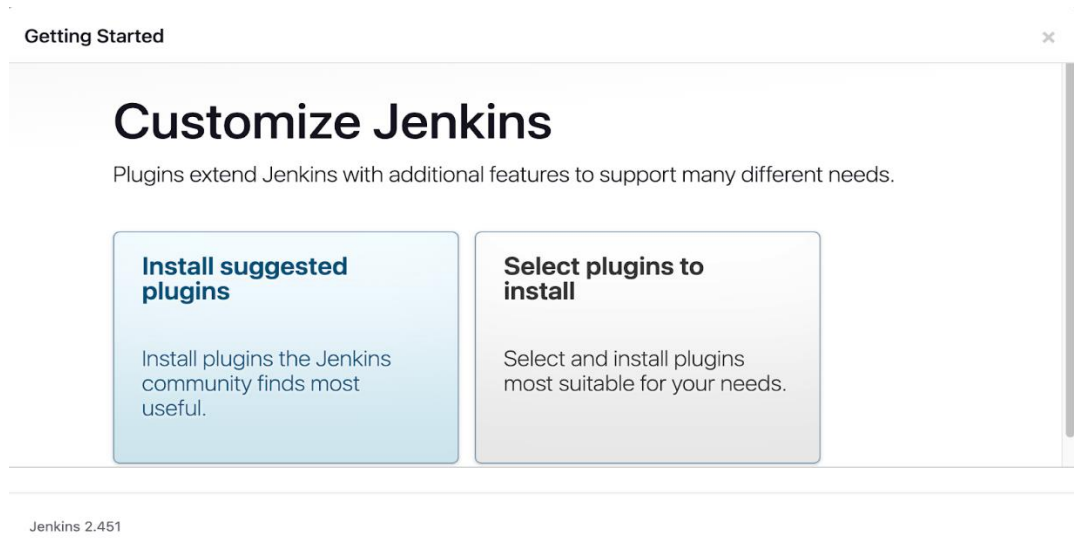
Administrator password

Continue

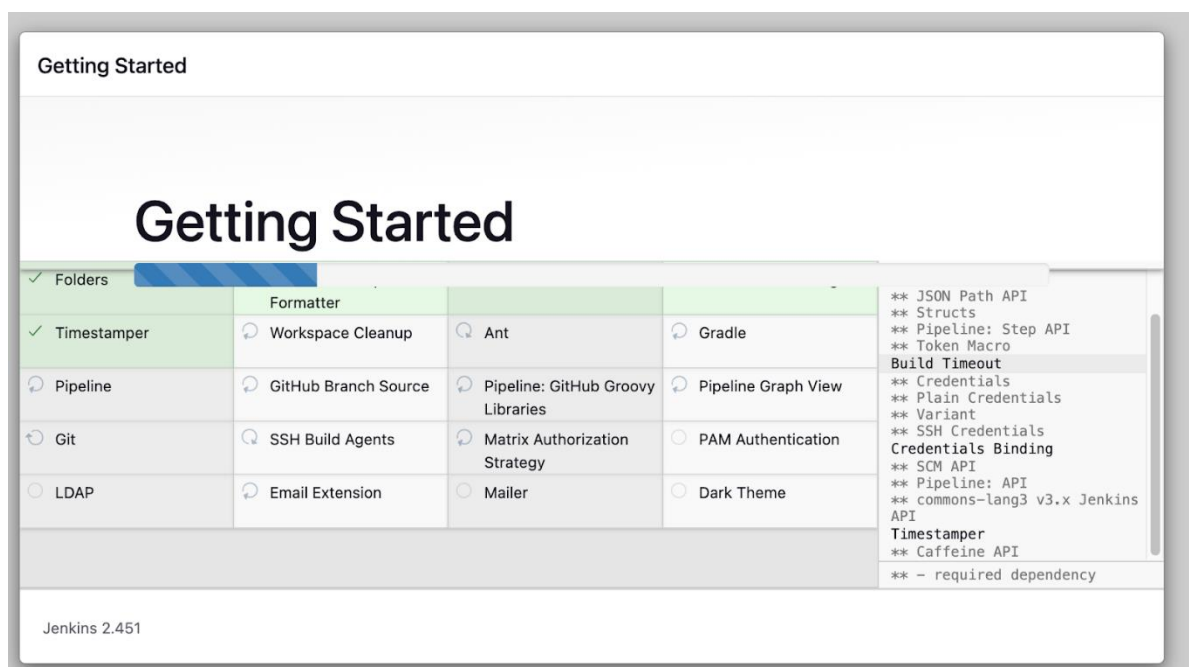
Initial Web Page of the setup flow



Password Retrieved from location provided.



Plugin Selection Screen



Plugins Installation Screen

Getting Started

Create First Admin User

Username

Password

Confirm password

Jenkins 2.451

Skip and continue as admin

Save and Continue

Admin User Creation

An admin user with a password will be created. From now on, utilize this to access the web user interface.

Getting Started

Instance Configuration

Jenkins URL:

http://localhost:8080/

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

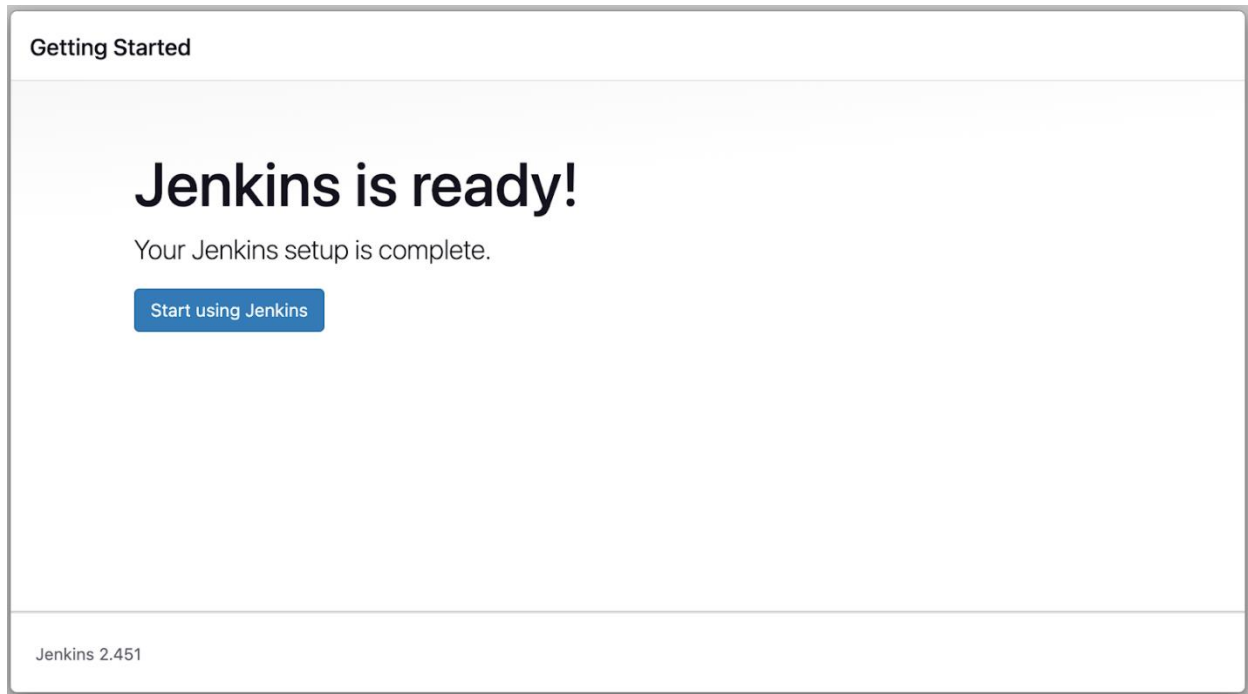
The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Jenkins 2.451

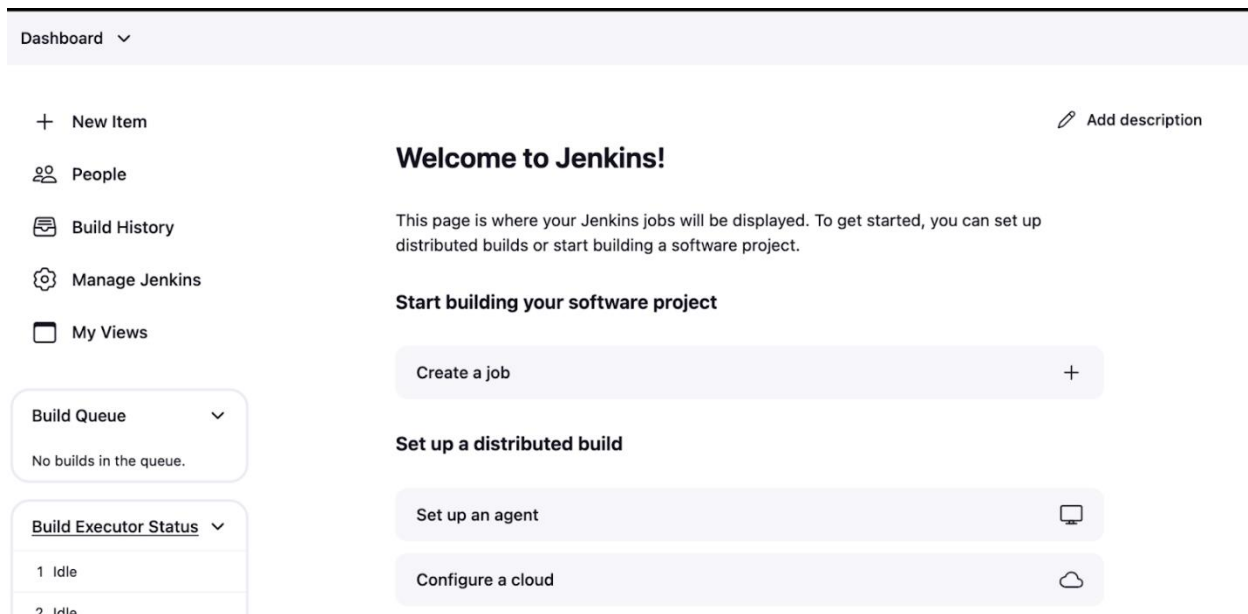
Not now

Save and Finish

Jenkins URL Configuration



Success Screen



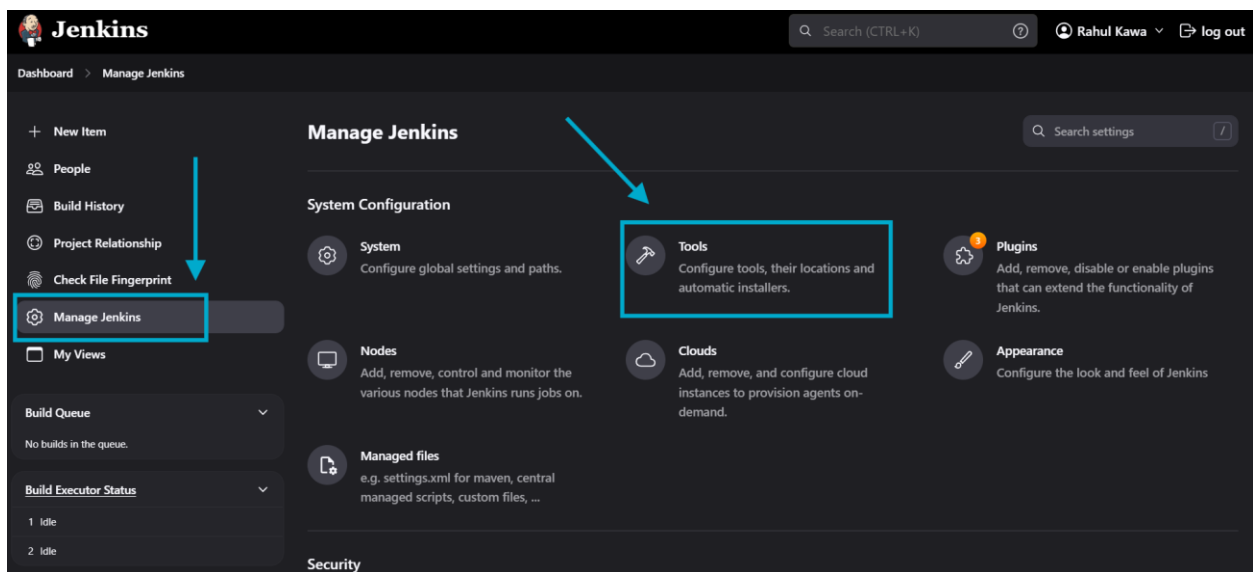
The above screen will appear if the process is successful. Jenkins can be used right now. When you select "**Start Using Jenkins**," the following page will appear.

Pipeline Creation

In Jenkins, a pipeline is a productive assembly line. It is a sequence of automated operations that generates and tests the code dynamically. Installing specific dependencies is the first step towards building a pipeline using Jenkins for the "**MavenProj**" (<https://github.com/raulkawa/ecommerce.git>) repository.

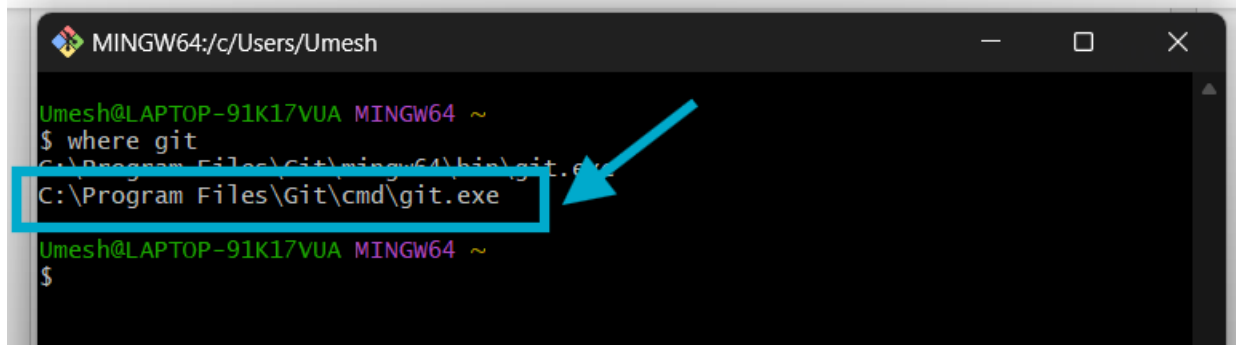
We require **Maven** and **Git** because our codebase is hosted on GitHub and written in Java.

1. Navigate to Manage **Jenkins** > **Tool**.

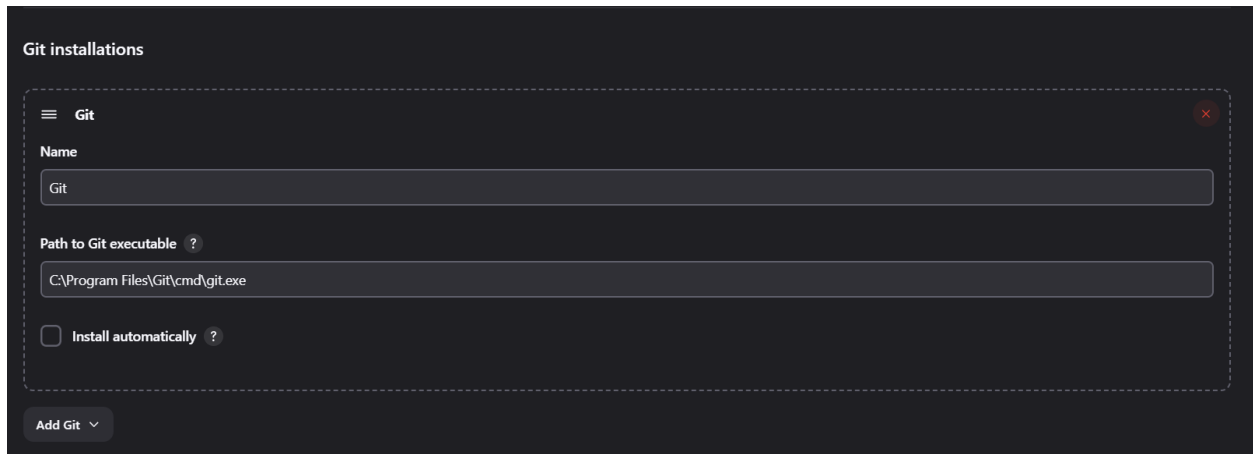


Next, since we are utilizing GIT as our version control tool, we must configure it. These actions must be taken to configure Git:

1. Go to the **Git** page from the Tool Configuration page.
2. To **Add Git**, click.
3. Installation Information:
 - a. Name: "**Git**"
 - b. How to Get a Git Executable Since our shell environment already had Git, we just used "**Git Bash**" with command "**where git**", it shows the path to our executable for Git.



```
MINGW64:/c/Users/Umesh
Umesh@LAPTOP-91K17VUA MINGW64 ~
$ where git
C:\Program Files\Git\mingw64\bin\git.exe
C:\Program Files\Git\cmd\git.exe
Umesh@LAPTOP-91K17VUA MINGW64 ~
$
```



Git installations

≡ Git

Name

Git

Path to Git executable ?

C:\Program Files\Git\cmd\git.exe

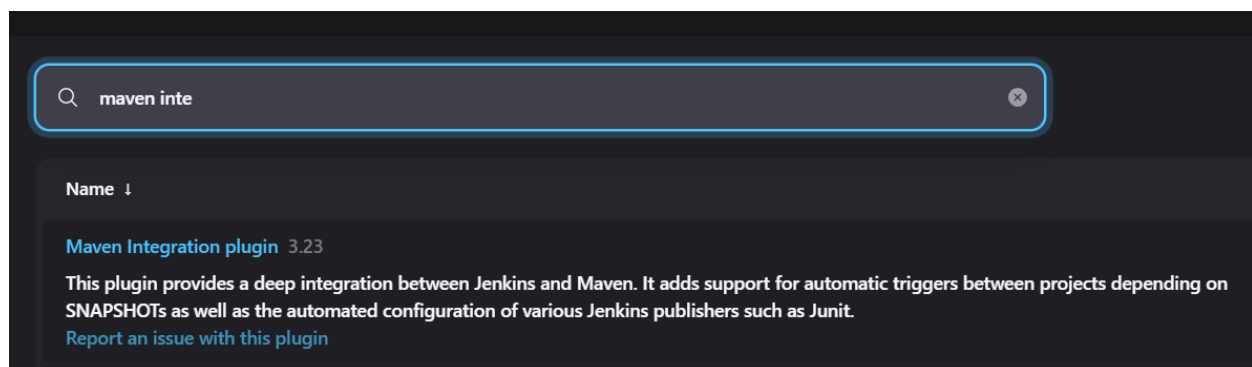
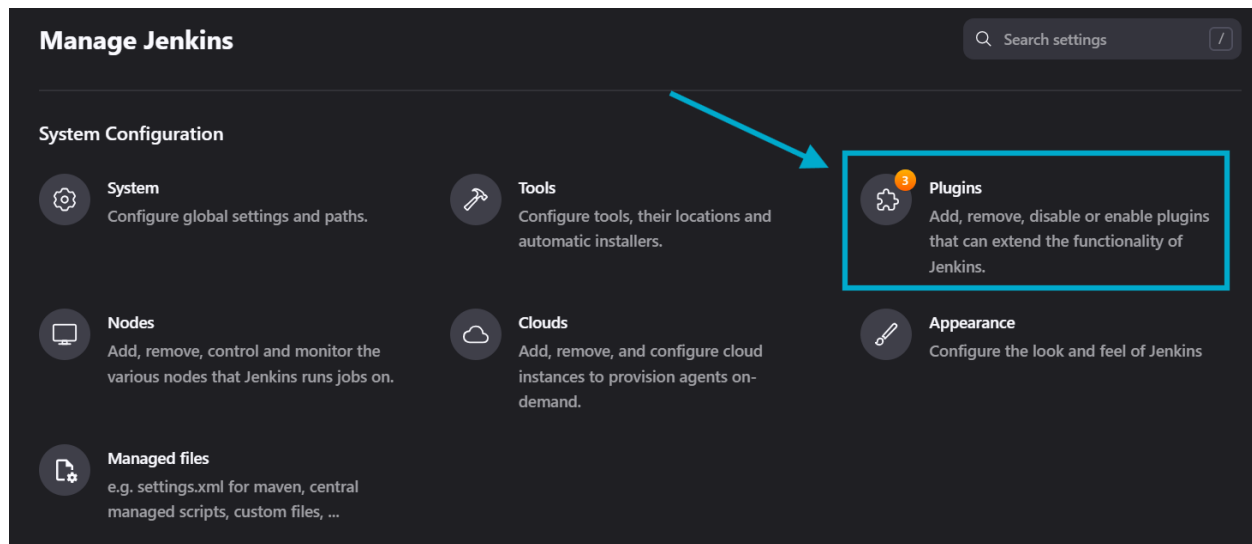
☐ Install automatically ?

Add Git ▾

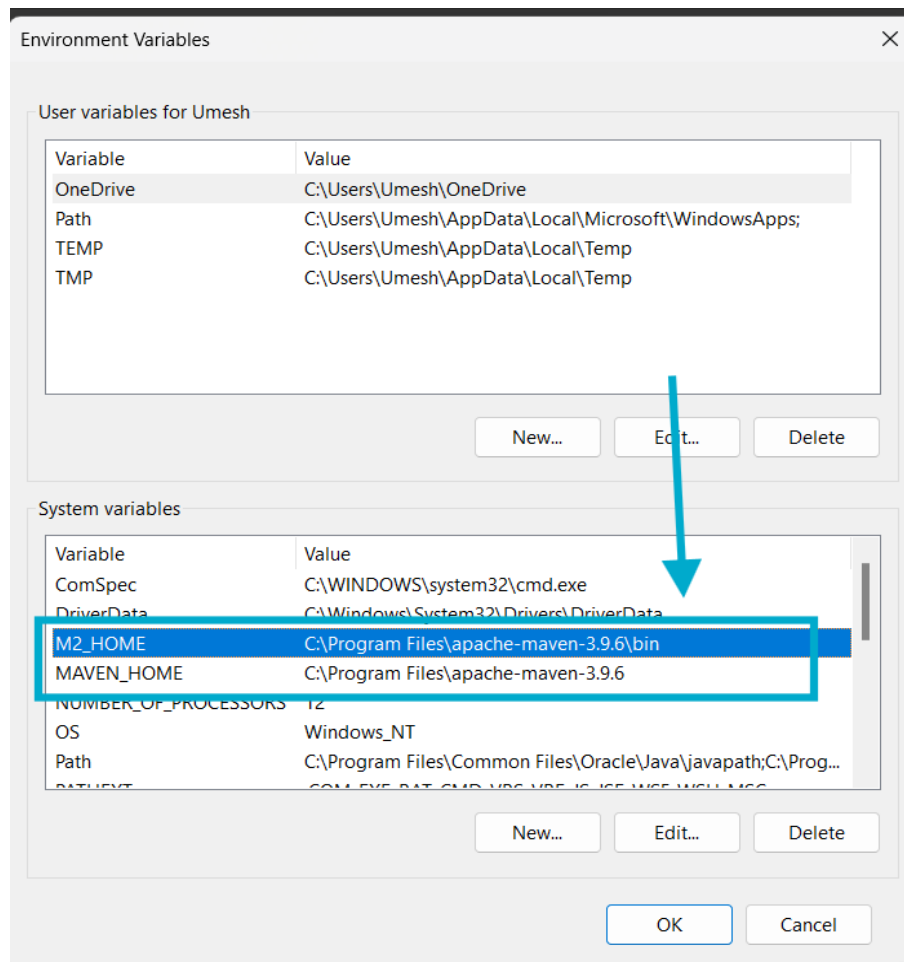
Git Configuration

Finally, a Maven configuration is required. However, we must install it because Maven does not have default plugins.

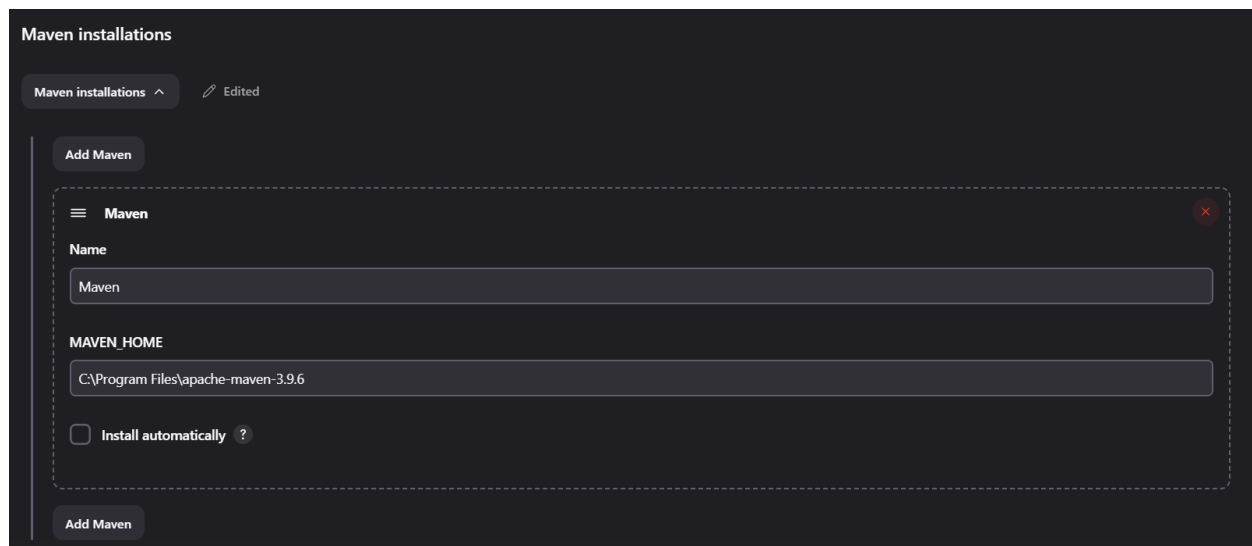
1. Go to **Manage Jenkins > Plugins**, look for the Maven plugin, and click Install to install and configure Maven.



2. Return to the page containing the **Tools**.
3. Locate the "**Maven**" Section on the Tools Page, then click **Add Maven**.
4. Installation details:
 - a. **Name:** As my system has **Maven**, I have used that as a name.
 - b. **MAVEN_HOME:** This is the path of my Maven installation directory.

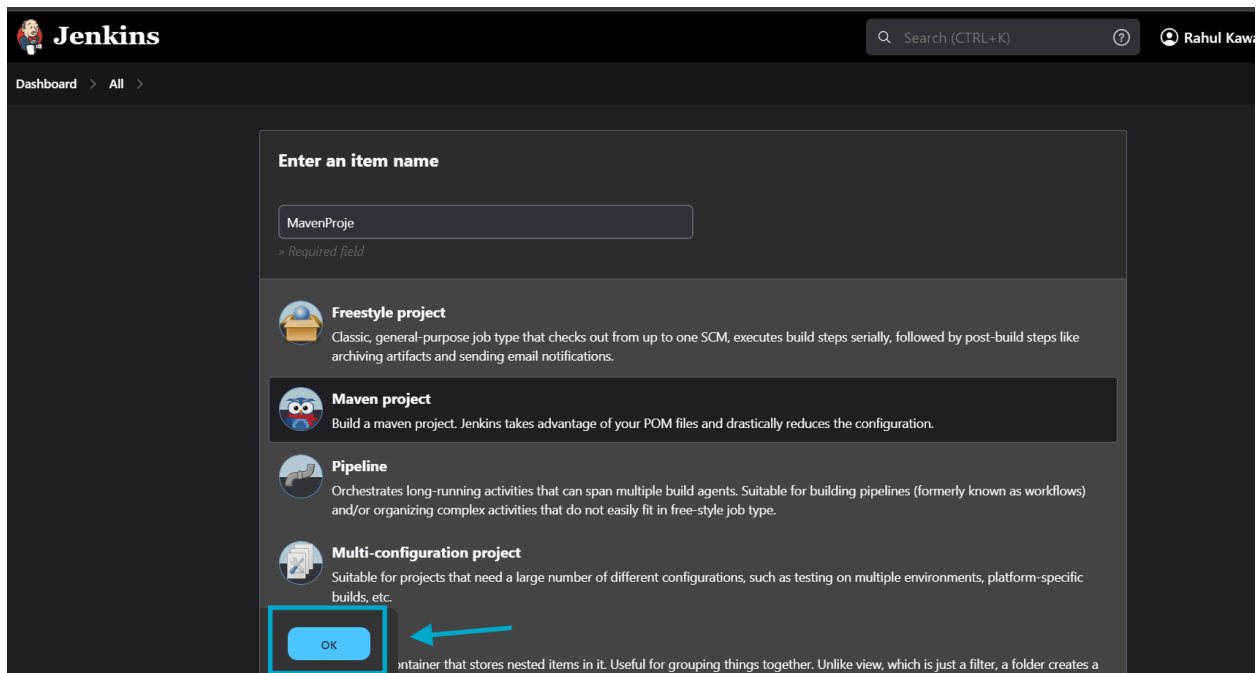


c. Uncheck "Install automatically"



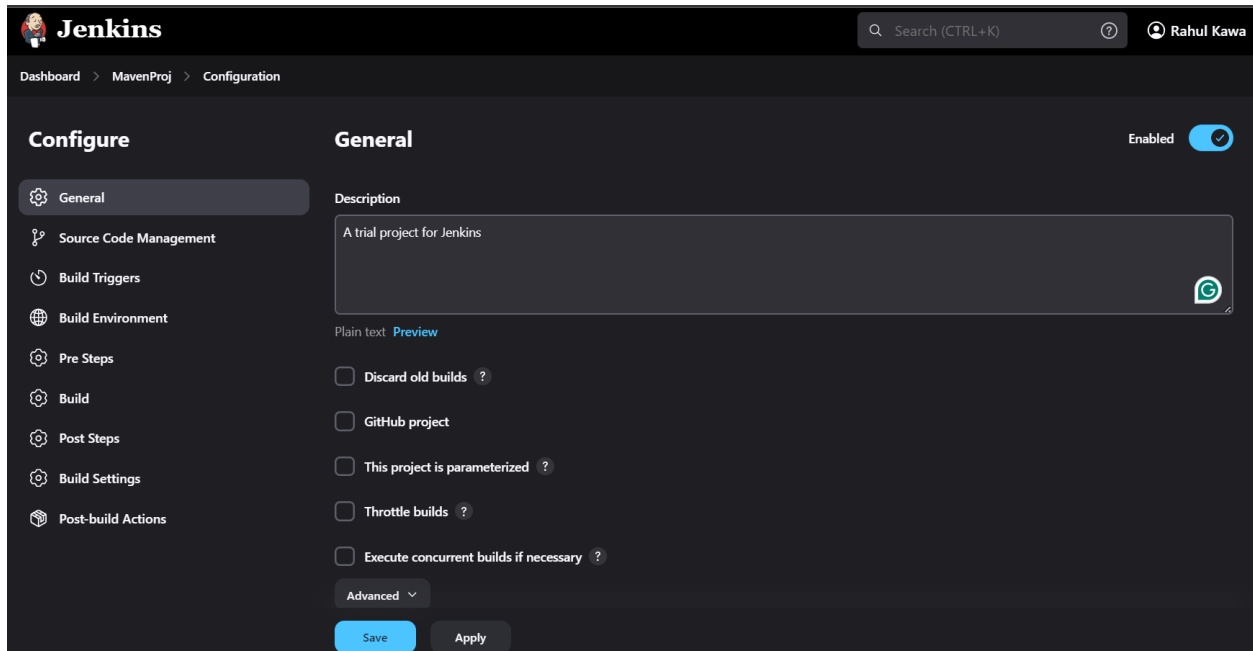
After everything is set up, we can begin the pipeline creation process. We will choose Maven as our project is based on the Maven framework. The steps are as follows:

1. Using the Jenkins dashboard, Select "**New Item.**"
2. Give the pipeline a name (such as "**MavenProj**").
3. Click "**OK**" after selecting the "**Maven**" project type.

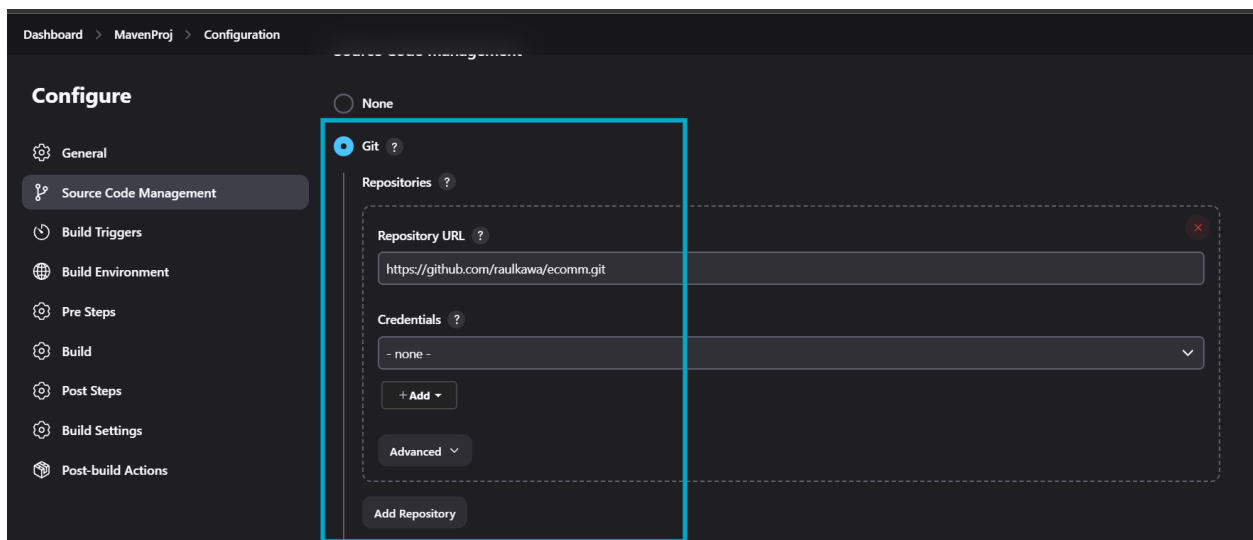


4. Configure Pipeline:

- a. **General:** We need to provide a description. Usually this is about the project.



- b. **Source Code Management:** Choose Git in this box, then enter the "Repository URL."



- c. **Build trigger:** Poll SCM will be used to pull changes from the repository at predefined intervals. We are going to use "**H/2 * * * ***." Jenkins will now be programmed to perform a two-minute check of the Git repository. The hash value H evenly distributes the load by making sure that numerous jobs with the same scheduling pattern don't run at precisely the same time.

Build Triggers

- ☒ Build whenever a SNAPSHOT dependency is built ?
- ☐ Schedule build when some upstream has no successful builds ?
- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☒ Poll SCM ?
 - Schedule ?
H/2 * * * *
- ☐ Ignore post-commit hooks ?

Would last have run at Tuesday, April 2, 2024, 4:28:38 PM Eastern Daylight Time; would next run at Tuesday, April 2, 2024, 4:30:38 PM Eastern Daylight Time.

Build Triggers Config

- d. The POM file needs to be configured for Maven projects.

Build

Root POM ?
pom.xml

Goals and options ?

Advanced ▾

Configuration of pom.xml

- e. To save this after everything is finished, click "**Save**."
- f. Now to manually start a build, we simply click "**Build Now**" on this.
- g. By going to the Console Output Page, we can view the **Console Output** of this build process.

```
Started by user Rahul Kawa
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\MavenProj
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/raulkawa/ecom.git
> C:\Program Files\Git\cmd\git.exe init C:\ProgramData\Jenkins\jenkins\workspace\MavenProj # timeout=10
Fetching upstream changes from https://github.com/raulkawa/ecom.git
> C:\Program Files\Git\cmd\git.exe --version # timeout=10
> git --version # 'git version 2.44.0.windows.1'
> C:\Program Files\Git\cmd\git.exe fetch --tags --force --progress -- https://github.com/raulkawa/ecom.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> C:\Program Files\Git\cmd\git.exe config remote.origin.url https://github.com/raulkawa/ecom.git # timeout=10
> C:\Program Files\Git\cmd\git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> C:\Program Files\Git\cmd\git.exe rev-parse "refs/remotes/origin/master^{commit}" # timeout=10
Checking out Revision c992c54bde6af51f67d8cfec5dba6cbda19f6c (refs/remotes/origin/master)
> C:\Program Files\Git\cmd\git.exe config core.sparsecheckout # timeout=10
> C:\Program Files\Git\cmd\git.exe checkout -f c992c54bde6af51f67d8cfec5dba6cbda19f6c # timeout=10
Commit message: "Added MIT LICENSE"
First time build. Skipping changelog.
Parsing POMs
Discovered a new module com.demo:shopiasy:pages:Maven:Mohann
```

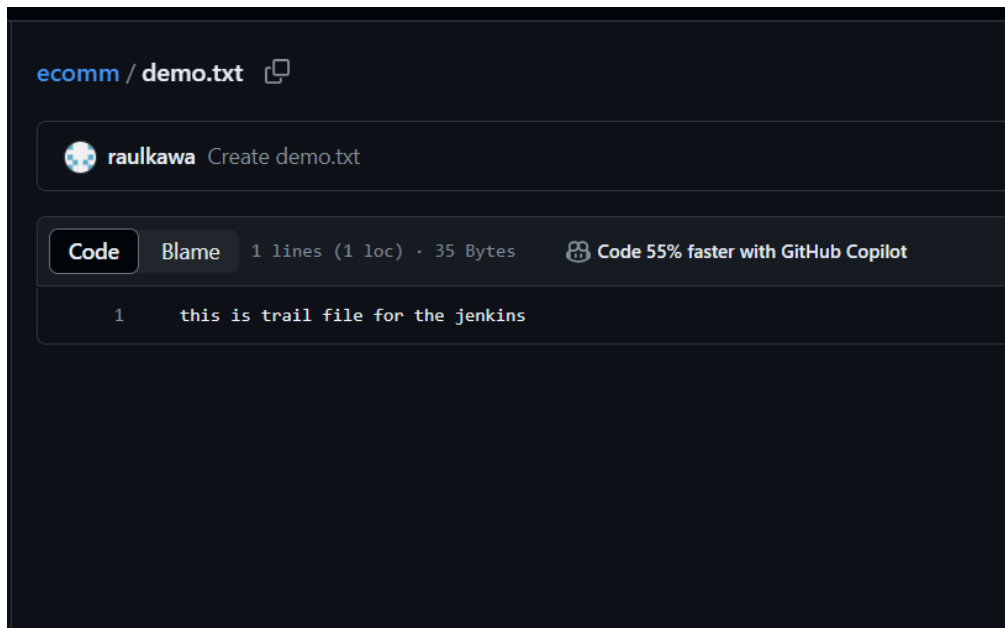
Console Output of Pipeline.

Build Success

Please take note that this build will rebuild every two-minute to see if there have been any changes to the remote GitHub repository.

Build With Git Repo Modifications:

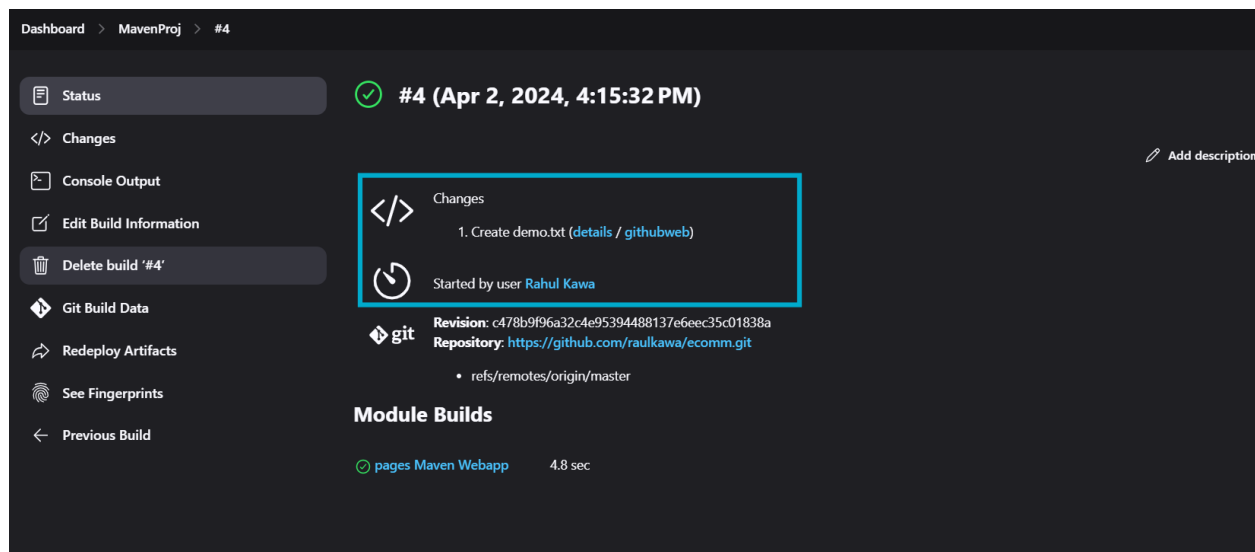
1. I have included a “**demo.txt**” file to test that the Git Changes were functioning as intended.



The screenshot shows a GitHub repository interface. At the top, it says "ecomm / demo.txt". Below that, there's a commit by "raulkawa" with the message "Create demo.txt". The file content is displayed as follows:

```
1  this is trail file for the jenkins
```

2. After adding, Rahul Kawa initiated a build that occurred shortly after.



Challenges

- **Configuring the environment:** It was challenging to ensure that JDK, Maven, and other components were set up correctly.

- **Plugin Conflicts:** Make sure all the plugins are compatible with each other's versions before installing a lot of them.
- **Local Port:** I had to confirm that no other application was using port 8080 and that it was available for use. Considering Jenkins's likely objection to this.

References

1. *Jenkins handbook*. (n.d.). Jenkins Handbook. <https://www.jenkins.io/doc/book/>
2. Porter, B., Van Zyl, J., & Lamy, O. (n.d.). *Maven – Welcome to Apache Maven*. <https://maven.apache.org/index.html>